

Laboratorio #3 — Redes Neuronales Recurrentes y LSTM (Sentimiento IMDB)

CC3092 Deep Learning y Sistemas Inteligentes

Ernesto Ascencio 23009

Repositorio: <https://github.com/AscencioSIUU/deepLearning/tree/main/labs/lab3-rnn-lstm>

1. Investigación de capas de PyTorch para RNN/ LSTM

`nn.Embedding(num_embeddings, embedding_dim, padding_idx)` mapea índices de token a vectores densos entrenables; `padding_idx` fija ese embedding en cero y sin gradiente. `nn.RNN(input_size, hidden_size, num_layers, batch_first, dropout)` implementa una RNN Elman ($h_t = \tanh(W_{ih} x_t + W_{hh} h_{t-1} + b)$) que devuelve salidas por paso y el último `h_n`. `nn.LSTM` tiene la misma firma pero añade un estado de celda `c_t` regulado por tres compuertas (forget/input/output) y devuelve (`h_n, c_n`). `pad_sequence` rellena una lista de tensores de longitud variable hasta el máximo del batch; `pack_padded_sequence/pad_packed_sequence` comprimen/descomprimen ese tensor para que la recurrente ignore el padding. `clip_grad_norm_(params, max_norm)` recorta la norma global del gradiente, previniendo exploding gradient. `nn.Dropout` en capas recurrentes de PyTorch solo actúa **entre** capas apiladas, por lo que se añade explícito sobre el último `h_n`.

Hidden vs. cell state: `h_t` es la salida expuesta en cada paso; `c_t` es memoria interna de la LSTM actualizada por suma ($c_t = f_{tc} c_{t-1} + i_{tg}_t$), filtrada por la compuerta de salida para producir `h_t`. **Vanishing/exploding gradient:** BPTT multiplica un jacobiano por cada paso temporal; valores propios <1 hacen el gradiente decaer exponencialmente (vanishing), >1 lo hacen explotar — el efecto se agrava con secuencias largas por la cantidad de factores multiplicados. **Por qué la LSTM mitiga esto:** su ruta de memoria es aditiva en vez de multiplicativa, dejando fluir el gradiente casi sin atenuarse a través de muchos pasos mientras las compuertas deciden qué olvidar/agregar/exponer.

2. Datos y arquitecturas

IMDB Reviews: 25,000 train / 25,000 test, 2 clases perfectamente balanceadas (12,500/12,500). Longitud de reseñas: min=4, max>2400, media≈233, mediana≈174 tokens (cola larga). Tokenización por regex `[a-z ']+` en minúsculas tras remover `<br />`; vocabulario de 20,000 tokens más frecuentes construido

solo sobre train (>97% de cobertura de ocurrencias), OOV→<unk>. Split adicional train→train/val 90/10 estratificado (22,500/2,500); test intocable hasta la evaluación final. max_len=300 para las arquitecturas principales (por encima de la mediana, acota el costo de padding/cómputo). MLP: bag-of-embeddings (promedio enmascarado) + capas Linear+ReLU+ Dropout configurables. RNN/ LSTM: nn.Embedding→recurrente many-to-one (con pack_padded_sequence)→Dropout→Linear. Entrenamiento con nn.CrossEntropyLoss y torch.optim.Adam, mismo loop (run_iteration/run_epoch) para las tres arquitecturas.

3. Tabla de resultados (16 iteraciones: 4 MLP + 4 RNN + 4 LSTM + 4 exp. de longitud)

#	Arq.	Descripción	Params	Val acc	Val F1	Tiempo
M1	MLP	[128], dropout=0 (mejor val)	2,013,186	0.8716	0.8715	5.7s
M2	MLP	[256,64], dropout=0.4, wd=1e-4	2,042,434	0.8700	0.8698	6.3s
M3	MLP	lr=0.1	2,013,186	0.8680	0.8680	5.7s
M4	MLP	emb=200, [128,64], dropout=0.3 (elegido final)	4,034,114	0.8680	0.8680	11.4s
R1	RNN	baseline	2,029,698	0.6368	0.6367	76.2s
R2	RNN	dropout=0.3, clip=5	2,029,698	0.6576	0.6575	77.3s
R3	RNN	lr=0.1 (peor)	2,029,698	0.5600	0.5598	77.3s
R4	RNN	emb=150, dropout=0.3, clip=5 (mejor)	3,036,098	0.7288	0.7284	109.8s
L1	LSTM	baseline	2,118,018	0.8324	0.8324	422.1s
L2	LSTM	dropout=0.3, clip=5	2,118,018	0.8256	0.8240	432.8s
L3	LSTM	lr=0.1 (peor)	2,118,018	0.5536	0.5521	418.7s
L4	LSTM	emb=150, dropout=0.3, clip=5 (mejor)	3,143,618	0.8588	0.8587	1705.6s

Las 4 iteraciones de MLP quedaron muy cerca entre sí (86.80%-87.16%, dentro del margen de ruido esperable con 2,500 ejemplos de validación). Se eligió **M4** como configuración final por su mejor balance capacidad/regularización, aunque M1 tuvo un val_acc marginalmente superior (diferencia de 0.36pp, no significativa). En RNN y LSTM, R4/L4 son inequívocamente las mejores.

Modelos finales (test, evaluados una única vez):

Arquitectura	Config	Params	Accuracy	Precision	Recall	F1	Tiempo
MLP	M4	4,034,114	0.8537	0.8542	0.8537	0.8536	11.4s
LSTM	L4	3,143,618	0.8454	0.8469	0.8454	0.8453	1705.6s

Arquitectura	Config	Params	Accuracy	Precision	Recall	F1	Tiempo
RNN	R4	3,036,098	0.7352	0.7366	0.7352	0.7348	109.8s

Experimento de longitud de secuencia (RNN vs. LSTM, mejor config c/u, 6 épocas):

Arq.	max_len	Val acc	Test acc	Test F1	Tiempo
RNN	50	0.5952	0.5897	0.5893	11.0s
RNN	400	0.7304	0.7456	0.7453	1952.8s
LSTM	50	0.7736	0.7456	0.7456	33.4s
LSTM	400	0.8388	0.8443	0.8443	6120.5s

4. Comparación de arquitecturas

El MLP tiene *más* parámetros (4.03M) que la LSTM (3.14M) o la RNN (3.04M) pero también el mejor accuracy de test — su capacidad extra viene de un vocabulario de embeddings más grande, no de profundidad recurrente. La LSTM logra un accuracy casi idéntico (-0.83pp) con **22% menos parámetros** que el MLP, evidencia de que sus compuertas usan la capacidad de forma más eficiente que la RNN, que con params similares a la LSTM (3.04M vs. 3.14M) queda **11.85pp** por debajo — la diferencia no es de tamaño sino de capacidad efectiva para propagar gradiente en 300 pasos. En tiempo de entrenamiento (CPU, este equipo): MLP << RNN << LSTM (11.4s / 109.8s / 1705.6s) — la LSTM tarda ~15x más que la RNN por sus 4 compuertas (4 matmults por paso) frente a la única transformación de la RNN simple; el MLP no tiene recurrencia y paraleliza todo el forward pass. Con 50 tokens ambas arquitecturas recurrentes parten de una base baja y similar (RNN test=0.590, LSTM test=0.746); al crecer a 400 tokens la RNN mejora pero solo alcanza lo que la LSTM ya lograba con 8x menos contexto — la RNN satura su capacidad de aprovechar contexto adicional mucho antes.

5. Discusión y análisis

Mayor impacto +/- por hiperparámetro: en el MLP ningún cambio tuvo impacto claramente positivo ni negativo (ni $\text{lr}=0.1$, M3) — el bag-of-embeddings tiene un techo de capacidad ~87% independiente de estos ajustes. En RNN y LSTM el patrón es claro: mayor impacto **positivo** = subir emb_dim + dropout + gradient clipping (R4: +9.2pp vs. baseline; L4: +2.6pp). Mayor impacto **negativo** en ambas = $\text{lr}=0.1$ sin clipping (R3: -7.7pp; L3: **-27.9pp**, casi random) — sin clip_grad_norm_, un paso 100x mayor desestabiliza el entrenamiento (exploding gradient), y afecta más a la LSTM por tener más parámetros a ajustar en su transformación.

Regularización: en el MLP, dropout+weight decay (M2) no superó al baseline (0.8700 vs. 0.8716) — el modelo no estaba lo bastante sobreajustado para beneficiarse. En la RNN, gradient clipping (R2 vs. R1: +2.1pp) sí ayudó — casi un requisito dado el exploding gradient. En la LSTM, dropout+clipping solos (L2, 6 épocas) quedaron ligeramente por debajo del baseline (0.8256 vs. 0.8324); sus compuertas ya regulan el flujo de gradiente razonablemente bien, así que el dropout adicional resta capacidad sin compensación en pocas épocas — la mejora final (L4) vino de más capacidad y más épocas, no solo de regularizar.

MLP vs. RNN vs. LSTM en test: el MLP obtuvo el mejor accuracy (85.37%), apenas por encima de la LSTM (84.54%) y muy por encima de la RNN (73.52%). Buena parte de la señal de sentimiento es léxica (“terrible”, “amazing”), y el promedio de embeddings ya la captura sin necesitar el orden de las palabras. La RNN simple sí intenta modelar el orden pero pierde señal de las primeras palabras por vanishing gradient en 300 pasos. La LSTM, gracias a sus compuertas, cierra casi toda la brecha con el MLP — validando que la memoria aditiva le permite competir con una representación puramente léxica, algo que la RNN simple no logra.

Experimento de longitud (4.1): contraintuitivamente la RNN no empeora al alargar la secuencia (50→400: val 0.595→0.730) — con poco contexto ninguna arquitectura tiene mucho que aprovechar. Lo relevante es el **techo**: a 400 tokens la LSTM (test=0.844) supera por mucho a la RNN (test=0.746), que apenas alcanza lo que la LSTM ya lograba con 50 tokens — exactamente lo esperado del vanishing gradient, que atenúa el gradiente de las posiciones lejanas de la RNN simple mientras la ruta aditiva de la LSTM sigue extrayendo señal de ellas.

Errores por tipo de reseña: el MLP comete más falsos negativos que falsos positivos (2,068 vs. 1,590 — lee positivas como negativas), mientras RNN y LSTM cometen más falsos positivos (RNN: 3,784 vs. 2,836; LSTM: 2,344 vs. 1,520 — leen negativas como positivas). Sin inspección de texto individual, el patrón es consistente con la intuición de dominio: reseñas con **sentimiento mixto o sarcasmo** son el caso clásico donde ni el promedio léxico ni el resumen del último estado oculto capturan bien cuál polaridad domina — la RNN, el modelo más débil, arrastra el doble de errores totales que la LSTM (6,620 vs. 3,864 de 25,000).

Modelo de producción: el **MLP** — gana en accuracy de test sobre los tres y es ~10x más rápido de entrenar que la RNN y ~150x más rápido que la LSTM en este equipo; su forward pass no tiene recurrencia, paraleliza completamente y no itera 300 pasos secuenciales por reseña en inferencia. La LSTM sería la alternativa razonable si se necesitara modelar negación/orden más allá del promedio léxico (segundo mejor accuracy), aceptando su costo notablemente mayor. La RNN simple no se justifica en ningún escenario aquí: peor accuracy que ambas y sigue siendo secuencial en inferencia, heredando el costo sin el beneficio. Con más datos y cómputo, un Transformer preentrenado (fine-tuning) probablemente superaría a los tres, pero para este dataset y con restricciones de eficiencia, el MLP bag-of-embeddings es la elección pragmática.